

1. Core Components of Agents

Component	Description
Task	A goal or objective given to the agent. It can be static (e.g., "translate this text") or dynamic (e.g., "monitor stock prices and act if a threshold is crossed").
Agent	An autonomous entity that perceives, reasons, and acts to accomplish its task. It may use rules, machine learning, large language models (LLMs), etc.
Tool	External capabilities the agent can use to complete its task, such as APIs, calculators, search engines, or other models.

 Additional components often include:

- **Memory:** To track context/history across steps
- **Planner:** For sequencing tasks
- **Executor:** For carrying out planned steps



2. Key Considerations in Multi-Agent Systems

Consideration

Explanation

✓ **Non-overlapping goals**

Each agent should have clearly scoped responsibilities to avoid redundancy.

✓ **Coordination**

Agents must be aware of each other's actions via communication or shared state.

✓ **Specialization**

Each agent should bring a unique skillset to the system.

✓ **Avoid infinite loops**

Prevent feedback loops (e.g., A calls B, B calls A forever).

✓ **Synchronization**

Some agents must wait for others to finish before acting.

✓ **Fault tolerance**

If one agent fails, others should continue or compensate.

3. Agent Communication Protocols

Type	Description
Formal Protocols	FIPA ACL: Structured messages (e.g., <code>inform</code> , <code>request</code> , <code>agree</code> , <code>refuse</code>) used in classical agent systems.
Modern Systems	Messaging queues like RabbitMQ , Kafka , or REST/GraphQL APIs between microservice-based agents.
LLM-Based Systems	Agents communicate via structured prompts (often JSON or natural language), e.g.,:

json

 Copiar  Editor

```
{ "role": "agent_a", "task": "summarize", "input": "..." }
```



4. Agent Orchestration + Metrics



Orchestration Techniques

Strategy	Description
Rule-based	Manually define the flow: "If A finishes, then trigger B." Simple but rigid.
Dynamic planning	A central planner (sometimes another agent) decides which agent to trigger based on runtime context.
Collaborative / decentralized	Agents interact autonomously to coordinate actions (used in LangGraph, CrewAI).



Orchestration Metrics

Metric	Meaning	
Tool usage effectiveness	% of tools used correctly out of those available.	
Average task completion time	Time taken per task by each agent.	
Collaborative success rate	% of tasks successfully completed through multi-agent collaboration.	
Redundancy rate	% of duplicate/overlapping tasks done by multiple agents.	
Overall task success	% of total tasks completed successfully.	
Communication efficiency	# of messages exchanged per successful task (ideally low).	

Example: LangGraph vs CrewAI

Scenario

You have two agents:

- "ResearchAgent: Gathers information from the web"
- "SummaryAgent: Summarizes the collected data"

Goal: Summarize recent information about "LangChain projects"

◆ LangGraph Example (Python)

python

📄 Copiar ✎ Editor

```
from langgraph.graph import StateGraph, END
from langchain.agents import initialize_agent, AgentType, Tool
from langchain.chat_models import ChatOpenAI
from langchain.tools import DuckDuckGoSearchRun
from typing import TypedDict

class AgentState(TypedDict):
    query: str
    info: str
    summary: str

# Tools
search_tool = DuckDuckGoSearchRun()
llm = ChatOpenAI(model="gpt-4")

# Agents
def research_agent(state):
    info = search_tool.run(state["query"])
    return {"query": state["query"], "info": info}
```

```
# Agents
```

```
def research_agent(state):
```

```
    info = search_tool.run(state["query"])
```

```
    return {"query": state["query"], "info": info}
```

```
def summary_agent(state):
```

```
    summary = llm.predict(f"Summarize the following:\n{state['info']}")
```

```
    return {"summary": summary}
```

```
# Build graph
```

```
builder = StateGraph(AgentState)
```

```
builder.add_node("Research", research_agent)
```

```
builder.add_node("Summarize", summary_agent)
```

```
builder.set_entry_point("Research")
```

```
builder.add_edge("Research", "Summarize")
```

```
builder.add_edge("Summarize", END)
```

```
graph = builder.compile()
```

```
# Run
```

```
result = graph.invoke({"query": "LangChain recent projects"})
```

```
print(result["summary"])
```




CrewAI Example (Python)

python

Copiar Editor

```
from crewai import Agent, Task, Crew
from langchain.tools import DuckDuckGoSearchRun
from langchain.chat_models import ChatOpenAI

search_tool = DuckDuckGoSearchRun()
llm = ChatOpenAI(model="gpt-4")

# Define agents
researcher = Agent(
    role="Researcher",
    goal="Collect fresh information about LangChain",
    tools=[search_tool],
    backstory="You're a skilled researcher with fast data retrieval skills.",
    llm=llm
)
```

```
# Define agents
researcher = Agent(
    role="Researcher",
    goal="Collect fresh information about LangChain",
    tools=[search_tool],
    backstory="You're a skilled researcher with fast data retrieval skills.",
    llm=llm
)

summarizer = Agent(
    role="Summarizer",
    goal="Summarize findings clearly",
    tools=[],
    backstory="You are a concise communicator.",
    llm=llm
)

# Define tasks
research_task = Task(
    description="Find the latest info on LangChain projects.",
    expected_output="A detailed report of LangChain's recent work.",
    agent=researcher
)
```

```
# Define tasks
research_task = Task(
    description="Find the latest info on LangChain projects.",
    expected_output="A detailed report of LangChain's recent work.",
    agent=researcher
)

summary_task = Task(
    description="Summarize the findings in a concise paragraph.",
    expected_output="A short, readable summary.",
    agent=summarizer
)

# Create Crew
crew = Crew(
    agents=[researcher, summarizer],
    tasks=[research_task, summary_task],
    verbose=True
)

result = crew.kickoff()
print(result)
```